

Experiment No: 09

Experiment Name: Traversing, Searching, Insertion and Deletion in a Doubly Linked List and Circular Linked List

Objectives:

- To implement Doubly Linked List and Circular Linked List and perform operations such as traversal, searching, insertion and deletion
- To understand dynamic memory management, pointer manipulation and optimized data structures for efficient data handling

Theory:

A Doubly Linked List is a linear dynamic data structure where each node contains data, a pointer to the next node and a pointer to the previous node. Traversal can be performed in both forward and backward directions

A Circular Linked List is a variation where the last node points back to the head, allowing circular traversal. These structures allow dynamic insertion and deletion at any position efficiently compared to static arrays

Algorithms:

1.1 | Traversal (Doubly Linked List):

- Start from head
- Move forward using next pointer and display each node
- Move backward from last node using prev pointer

1.2 | Traversal (Circular Linked List):

- Start from head
- Move forward using next until reaching head again

2 | Searching:

- Start from head
- Compare each node's data with the target
- If found, display the position; otherwise, continue

3 | Insertion:

- Beginning: Update head and node pointers
- End: Traverse to last node, update pointers
- Specific Position: Traverse and adjust pointers to insert

4 | Deletion:

- Beginning: Update head, adjust pointers, free node
- End: Traverse to last node, update previous node's pointer, free node
- Specific Position: Traverse, update adjacent nodes, free node

Code:

Doubly Linked List:

```
G++ DLLcpp > ...
1  #include <iostream>
2  using namespace std;
3
4  // Node structure for Doubly Linked List
5  struct Node
6  {
7      int data;
8      Node *next;
9      Node *prev;
10 };
11
12 Node *head = nullptr;
13
14 // Traverse in Doubly Linked List
15 void traverse()
16 {
17     Node *ptr = head;
18     if (ptr == nullptr)
19     {
20         cout << "List is empty\n";
21         return;
22     }
23     cout << "Forward: ";
24     while (ptr != nullptr)
25     {
26         cout << ptr->data << " ";
27         if (ptr->next == nullptr)
28             break;
29         ptr = ptr->next;
30     }
31     cout << "\nBackward: ";
32     while (ptr != nullptr)
33     {
34         cout << ptr->data << " ";
35         ptr = ptr->prev;
36     }
37     cout << "\n";
38 }
39
40 // Search in Doubly Linked List
41 void search(int key)
42 {
43     Node *ptr = head;
44     int pos = 0;
45     bool found = false;
46     while (ptr != nullptr)
47     {
48         if (ptr->data == key)
49         {
50             cout << "Element " << key << " found at position " << pos << "\n";
51             found = true;
52             break;
53         }
54         ptr = ptr->next;
55         pos++;
56     }
57     if (!found)
58         cout << "Element " << key << " not found\n";
59 }
60
61 // Insert at end of Doubly Linked List
62 void insertEnd(int val)
63 {
64     Node *newNode = new Node[val, nullptr, nullptr];
65     if (!head)
66     {
67         head = newNode;
68         return;
69     }
70     Node *ptr = head;
71     while (ptr->next != nullptr)
72         ptr = ptr->next;
73     ptr->next = newNode;
74     newNode->prev = ptr;
75 }
76
77 // Delete from end of Doubly Linked List
78 void deleteEnd()
79 {
80     if (!head)
81     {
82         cout << "List is empty\n";
83         return;
84     }
85     Node *ptr = head;
86     if (!ptr->next)
87     {
88         delete ptr;
89         head = nullptr;
90         return;
91     }
92     while (ptr->next != nullptr)
93         ptr = ptr->next;
94     ptr->prev->next = nullptr;
95     delete ptr;
96 }
97
98 int main()
99 {
100     int choice, val, key;
101     while (true)
102     {
103         cout << "\n~~~~~ Doubly Linked List Operations ~~~~\n";
104         cout << "1. Traversal\n2. Search\n3. Insert at End\n4. Delete from End\n5. Exit\n";
105         cout << "Enter your choice: ";
106         cin >> choice;
107         switch (choice)
108         {
109             case 1:
110                 traverse();
111                 break;
112             case 2:
113                 cout << "Enter element to search: ";
114                 cin >> key;
115                 search(key);
116                 break;
117             case 3:
118                 cout << "Enter value to insert: ";
119                 cin >> val;
120                 insertEnd(val);
121                 break;
122             case 4:
123                 deleteEnd();
124                 break;
125             case 5:
126                 return 0;
127             default:
128                 cout << "Invalid choice! Try again\n";
129         }
130     }
131 }
```

Circular Linked List:

```
(G+ CLcpp > ...
1 #include <iostream>
2 using namespace std;
3
4 // Node structure for Circular Linked List
5 struct Node
6 {
7     int data;
8     Node *next;
9     Node *prev;
10 };
11
12 Node *head = nullptr;
13
14 // Traverse in Circular Linked List
15 void traverse()
16 {
17     if (!head)
18     {
19         cout << "List is empty\n";
20         return;
21     }
22     Node *ptr = head;
23     cout << "Forward: ";
24     do
25     {
26         cout << ptr->data << " ";
27         ptr = ptr->next;
28     } while (ptr != head);
29     cout << "\nBackward: ";
30     ptr = head->prev; // start from last node
31     Node *start = ptr;
32     do
33     {
34         cout << ptr->data << " ";
35         ptr = ptr->prev;
36     } while (ptr != start);
37     cout << "\n";
38 }
39
40 // Search in Circular Linked List
41 void search(int key)
42 {
43     if (!head)
44     {
45         cout << "List is empty\n";
46         return;
47     }
48     Node *ptr = head;
49     int pos = 0;
50     bool found = false;
51     do
52     {
53         if (ptr->data == key)
54         {
55             cout << "Element " << key << " found at position " << pos << "\n";
56             found = true;
57             break;
58         }
59         ptr = ptr->next;
60         pos++;
61     } while (ptr != head);
62     if (!found)
63         cout << "Element " << key << " not found\n";
64 }
65
66 // Insert at end of Circular Linked List
67 void insertEnd(int val)
68 {
69     Node *newNode = new Node(val, nullptr, nullptr);
70     if (!head)
71     {
72         head = newNode;
73         head->next = head;
74         head->prev = head;
75     }
76     else
77     {
78         Node *tail = head->prev; // Last node
79         tail->next = newNode;
80         newNode->prev = tail;
81         head->prev = newNode;
82     }
83
84 // Delete from end of Circular Linked List
85 void deleteEnd()
86 {
87     if (!head)
88     {
89         cout << "List is empty\n";
90         return;
91     }
92     if (head->next == head)
93     { // only one node
94         delete head;
95         head = nullptr;
96         return;
97     }
98     Node *tail = head->prev;
99     tail->prev->next = head;
100     head->prev = tail->prev;
101     delete tail;
102 }
103
104 int main()
105 {
106     int choice, val, key;
107     while (true)
108     {
109         cout << "\n==== Circular Linked List Operations =====\n";
110         cout << "1. Traversal\n2. Search\n3. Insert at End\n4. Delete from End\n5. Exit\n";
111         cout << "Enter your choice: ";
112         cin >> choice;
113         switch (choice)
114         {
115             case 1:
116                 traverse();
117                 break;
118             case 2:
119                 cout << "Enter element to search: ";
120                 cin >> key;
121                 search(key);
122                 break;
123             case 3:
124                 cout << "Enter value to insert: ";
125                 cin >> val;
126                 insertEnd(val);
127                 break;
128             case 4:
129                 deleteEnd();
130                 break;
131             case 5:
132                 return 0;
133             default:
134                 cout << "Invalid choice! Try again\n";
135         }
136     }
137 }
```

Procedure:

- VS Code was opened and a new project was created
- The Doubly Linked List code was written, compiled and syntax errors fixed
- Menu driven operations such as Traversal, Search, Insert, Delete were performed and changes were verified after each action
- Forward and backward traversal was observed to ensure correctness and memory was freed when nodes were deleted
- The Doubly Linked List code was extended to the Circular Linked List by linking the last node to head and updating head->prev
- The Circular Linked List operations were tested, ensuring traversal, insertion and deletion maintained proper circular links

Results:

Doubly Linked List:

```

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 1

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 2

```

```

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 3

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 1
Forward: 1 2 3
Backward: 3 2 1

```

```

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 2
Enter element to search: 3
Element 3 found at position 2

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 4

```

```

~~~ Doubly Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 1
Forward: 1 2
Backward: 2 1

```

Circular Linked List:

```

~~~ Circular Linked List operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 1

~~~ Circular Linked List operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 2

```

```

~~~ Circular Linked List operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 3
Enter value to insert: 3

~~~ Circular Linked List operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 1
Forward: 1 2 3
Backward: 3 2 1

```

```

~~~ Circular linked List operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 2
Enter element to search: 3
Element 3 found at position 2

~~~ Circular Linked List operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 4

```

```

~~~ Circular Linked List Operations ~~~
1. Traversal
2. Search
3. Insert at End
4. Delete from End
5. Exit
Enter your choice: 1
Forward: 1 2
Backward: 2 1

```

Complexity Analysis:

Operation	Time Complexity	Space Complexity
Traversal (DLL/CLL)	$O(n)$	$O(1)$
Searching	$O(n)$	$O(1)$
Insertion at beginning	$O(1)$	$O(1)$ per node
Insertion at end	$O(n)$	$O(1)$ per node
Deletion at beginning	$O(1)$	$O(1)$
Deletion at end	$O(n)$	$O(1)$

Precautions:

- Always checked for NULL pointers before accessing nodes
- Validated positions for insertion and deletion to avoid segmentation faults
- Freed memory of deleted nodes to prevent memory leaks
- Avoided negative or invalid indices during operations

Limitations:

- Random access is inefficient compared to arrays
- Operations require pointer manipulation, which may lead to errors if not careful

Conclusion:

This experiment provided hands on experience in implementing and manipulating Doubly Linked Lists and understanding the logic for Circular Linked Lists. By completing this lab, we gain a solid understanding of advanced pointer based dynamic data structures, foundational for stacks, queues and complex structures like graphs

Experiment No: 10

Experiment Name: Implementing and Analyzing Merge Sort and Quick Sort Algorithms

Objectives:

- To implement Merge Sort and Quick Sort using C++
- To understand divide-and-conquer strategy in sorting algorithms
- To analyze time and space complexity of efficient sorting techniques

Theory:

Sorting is one of the most fundamental operations in computer science used to arrange data in a structured order, typically ascending or descending. It plays a crucial role in optimizing searching, data processing, and overall algorithm efficiency. Efficient sorting techniques are essential when dealing with large datasets, as they significantly reduce computational overhead in downstream operations

1 | Merge Sort

Merge Sort is a classic divide-and-conquer algorithm that works by recursively dividing the array into two halves until each sub-array contains a single element. These sub-arrays are then merged in a sorted manner to produce the final sorted array. The key idea behind Merge Sort is that merging two already sorted arrays is efficient and predictable. Because it always divides the problem into equal halves, its performance remains stable across all input cases

2 | Quick Sort

Quick Sort is another powerful divide-and-conquer sorting algorithm that selects a pivot element and partitions the array such that elements smaller than the pivot move to the left and larger elements move to the right. The same process is recursively applied to the sub-arrays. Unlike Merge Sort, Quick Sort does not require additional memory for merging, making it more space-efficient. However, its performance depends heavily on pivot selection, and poor pivot choices can degrade its efficiency

Algorithms:

1 | Merge Sort Algorithm:

- If the array contains more than one element, divide it into two halves
- Recursively apply Merge Sort on the left half
- Recursively apply Merge Sort on the right half
- Merge both sorted halves into a single sorted array by comparing elements sequentially
- Return the fully sorted array

2 | Quick Sort Algorithm:

- Select the last element of the array as pivot
- Initialize a pointer for smaller element region
- Traverse the array and compare each element with pivot
- Swap elements to ensure smaller elements are placed on the left side
- Place pivot in its correct sorted position
- Recursively apply Quick Sort on left and right partitions
- Stop when sub-array size becomes 0 or 1

Code:

Merge Sort:

```
G+ mergeSort.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  void merge(int arr[], int l, int m, int r)
5  {
6      int n1 = m - l + 1;
7      int n2 = r - m;
8
9      int L[n1], R[n2];
10
11     for (int i = 0; i < n1; i++)
12         L[i] = arr[l + i];
13     for (int j = 0; j < n2; j++)
14         R[j] = arr[m + 1 + j];
15
16     int i = 0, j = 0, k = l;
17
18     while (i < n1 && j < n2)
19     {
20         if (L[i] <= R[j])
21             arr[k++] = L[i++];
22         else
23             arr[k++] = R[j++];
24     }
25
26     while (i < n1)
27         arr[k++] = L[i++];
28     while (j < n2)
29         arr[k++] = R[j++];
30 }
31
32 void mergeSort(int arr[], int l, int r)
33 {
34     if (l < r)
35     {
36         int m = l + (r - l) / 2;
37         mergeSort(arr, l, m);
38         mergeSort(arr, m + 1, r);
39         merge(arr, l, m, r);
40     }
41 }
42
43
44 int main()
45 {
46     int n;
47     cout << "Enter number of elements: ";
48     cin >> n;
49
50     int arr[n];
51
52     cout << "Enter elements: ";
53     for (int i = 0; i < n; i++)
54         cin >> arr[i];
55
56     mergeSort(arr, 0, n - 1);
57
58     cout << "Sorted array (Merge Sort): ";
59     for (int i = 0; i < n; i++)
60         cout << arr[i] << " ";
61
62     return 0;
63 }
```

Quick Sort:

```
G+ quickSort.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  int partition(int arr[], int low, int high)
5  {
6      int pivot = arr[high];
7      int i = low - 1;
8
9      for (int j = low; j < high; j++)
10     {
11         if (arr[j] <= pivot)
12         {
13             i++;
14             swap(arr[i], arr[j]);
15         }
16     }
17     swap(arr[i + 1], arr[high]);
18     return i + 1;
19 }
20
21 void quickSort(int arr[], int low, int high)
22 {
23     if (low < high)
24     {
25         int pi = partition(arr, low, high);
26
27         quickSort(arr, low, pi - 1);
28         quickSort(arr, pi + 1, high);
29     }
30 }
31
32
33 int main()
34 {
35     int n;
36     cout << "Enter number of elements: ";
37     cin >> n;
38
39     int arr[n];
40
41     cout << "Enter elements: ";
42     for (int i = 0; i < n; i++)
43         cin >> arr[i];
44
45     quickSort(arr, 0, n - 1);
46
47     cout << "Sorted array (Quick Sort): ";
48     for (int i = 0; i < n; i++)
49         cout << arr[i] << " ";
50
51     return 0;
52 }
```

Procedure:

1. Created two separate C++ files for Merge Sort and Quick Sort implementation
2. Implemented both algorithms using divide-and-conquer logic
3. Compiled and executed programs successfully in a C++ IDE (VS Code)
4. Provided sample input arrays and observed sorted outputs
5. Verified correctness by comparing results with manual sorting

Results:

```
Enter number of elements: 5          Enter number of elements: 5
Enter elements: 5 3 4 1 2          Enter elements: 5 3 4 1 2
Sorted array (Merge Sort): 1 2 3 4 5  Sorted array (Quick Sort): 1 2 3 4 5
```

Complexity Analysis:

Operation	Merge Sort	Quick Sort
Best Case	$O(n \log n)$	$O(n \log n)$
Average Case	$O(n \log n)$	$O(n \log n)$
Worst Case	$O(n \log n)$	$O(n^2)$
Space Complexity	$O(n)$	$O(\log n)$

Precautions:

- Ensure array indices remain within valid range
- Avoid stack overflow by testing small inputs first
- Carefully handle recursion base conditions
- Ensure correct pivot handling in Quick Sort
- Verify input validity before processing

Limitations:

- Merge Sort requires additional memory for temporary arrays
- Quick Sort performance depends on pivot selection strategy
- Recursive calls may cause overhead for large datasets

Conclusion:

This experiment provides practical implementation and analysis of Merge Sort and Quick Sort algorithms using C++. It enhances understanding of divide-and-conquer strategy, recursion, and algorithm efficiency. Merge Sort ensures consistent performance across all cases, while Quick Sort offers better practical performance with efficient in-place sorting behavior.

Experiment No: 11

Experiment Name: Traversing Binary Tree, Binary Search Tree, Insertion and Deletion on Binary Search Tree

Objectives:

- To implement Binary Tree traversal techniques (Inorder, Preorder and Postorder)
- To implement Binary Search Tree operations such as insertion and deletion

Theory:

A Binary Tree (BT) is a hierarchical data structure where each node has at most two children: left and right. It does not enforce any ordering on values

Tree traversal techniques are used to visit all nodes:

- Inorder (Left $\rightarrow$ Root $\rightarrow$ Right)
- Preorder (Root $\rightarrow$ Left $\rightarrow$ Right)
- Postorder (Left $\rightarrow$ Right $\rightarrow$ Root)

On the other hand, a Binary Search Tree (BST) is a special type of binary tree where:

- Left subtree contains smaller values than root
- Right subtree contains larger values than root

Algorithms:

1. Binary Tree Traversal:

1.1 | Inorder (Left $\rightarrow$ Parent $\rightarrow$ Right)

1.2 | Preorder (Parent $\rightarrow$ Left $\rightarrow$ Right)

1.3 | Postorder (Left $\rightarrow$ Right $\rightarrow$ Parent)

2. Binary Search Tree:

2.1 | Insertion:

- Start from root
- If tree is empty, create root node
- Compare value:
 - If smaller $\rightarrow$ go left
 - If larger $\rightarrow$ go right
- Insert at correct position

2.1 | Deletion:

- Search for node
 - Case 1: No child $\rightarrow$ delete directly
 - Case 2: One child $\rightarrow$ replace with child
 - Case 3: Two children $\rightarrow$ replace with inorder successor

Code:

1. Binary Tree Traversal

1.1 | Inorder

```
Lab 8 > G+ 1_inorder.cpp > inorder(Node *)
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Node
5  {
6  public:
7      int data;
8      Node *left;
9      Node *right;
10     Node(int val)
11     {
12         data = val;
13         left = right = NULL;
14     }
15 };
16
17 Node *buildTreeFromFile(string filename)
18 {
19     ifstream file(filename);
20     int l[100], r[100], v[100];
21     Node *nodes[101];
22     int n = 0;
23
24     while (true)
25     {
26         int li, vi, ri;
27         file >> li >> vi >> ri;
28         if (li == -1 && vi == -1 && ri == -1)
29             break;
30         l[n] = li;
31         v[n] = vi;
32         r[n] = ri;
33         n++;
34     }
35
36     for (int i = 1; i <= n; i++)
37         nodes[i] = new Node(v[i - 1]);
38     for (int i = 1; i <= n; i++)
39     {
40         if (l[i - 1] != 0)
41             nodes[i]->left = nodes[l[i - 1]];
42         if (r[i - 1] != 0)
43             nodes[i]->right = nodes[r[i - 1]];
44     }
45
46     return nodes[1];
47 }
48
49 void inorder(Node *root)
50 {
51     stack<Node *> st;
52     Node *curr = root;
53
54     while (curr != NULL || !st.empty())
55     {
56         while (curr != NULL)
57         {
58             st.push(curr);
59             curr = curr->left;
60         }
61         curr = st.top();
62         st.pop();
63         cout << curr->data << " ";
64         curr = curr->right;
65     }
66 }
67
68 int main()
69 {
70     Node *root = buildTreeFromFile("file.txt");
71     cout << "Inorder: ";
72     inorder(root);
73     cout << endl;
74     return 0;
75 }
```

1.2 | Preorder

```
Lab 8 > G+ 2_preorder.cpp > main()
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Node
5  {
6  public:
7      int data;
8      Node *left;
9      Node *right;
10     Node(int val)
11     {
12         data = val;
13         left = right = NULL;
14     }
15 };
16
17 Node *buildTreeFromFile(string filename)
18 {
19     ifstream file(filename);
20     int l[100], r[100], v[100];
21     Node *nodes[101];
22     int n = 0;
23
24     while (true)
25     {
26         int li, vi, ri;
27         file >> li >> vi >> ri;
28         if (li == -1 && vi == -1 && ri == -1)
29             break;
30         l[n] = li;
31         v[n] = vi;
32         r[n] = ri;
33         n++;
34     }
35
36     for (int i = 1; i <= n; i++)
37         nodes[i] = new Node(v[i - 1]);
38     for (int i = 1; i <= n; i++)
39     {
40         if (l[i - 1] != 0)
41             nodes[i]->left = nodes[l[i - 1]];
42         if (r[i - 1] != 0)
43             nodes[i]->right = nodes[r[i - 1]];
44     }
45
46     return nodes[1];
47 }
48
49 void preorder(Node *root)
50 {
51     if (!root)
52         return;
53     stack<Node *> st;
54     st.push(root);
55
56     while (!st.empty())
57     {
58         Node *curr = st.top();
59         st.pop();
60         cout << curr->data << " ";
61
62         if (curr->right)
63             st.push(curr->right);
64         if (curr->left)
65             st.push(curr->left);
66     }
67 }
68
69 int main()
70 {
71     Node *root = buildTreeFromFile("file.txt");
72     cout << "Preorder: ";
73     preorder(root);
74     cout << endl;
75     return 0;
76 }
```

1.3 | Postorder

Lab 8 > G+ 3_postorder.cpp > ...

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Node
5  {
6  public:
7      int data;
8      Node *left;
9      Node *right;
10     Node(int val)
11     {
12         data = val;
13         left = right = NULL;
14     }
15 };
16
17 Node *buildTreeFromFile(string filename)
18 {
19     ifstream file(filename);
20     int l[100], r[100], v[100];
21     Node *nodes[101];
22     int n = 0;
23
24     while (true)
25     {
26         int li, vi, ri;
27         file >> li >> vi >> ri;
28         if (li == -1 && vi == -1 && ri == -1)
29             break;
30         l[n] = li;
31         v[n] = vi;
32         r[n] = ri;
33         n++;
34     }
35
36     for (int i = 1; i <= n; i++)
37         nodes[i] = new Node(v[i - 1]);
38     for (int i = 1; i <= n; i++)
39     {
40         if (l[i - 1] != 0)
41             nodes[i]->left = nodes[l[i - 1]];
42         if (r[i - 1] != 0)
43             nodes[i]->right = nodes[r[i - 1]];
44     }
45     return nodes[1];
46 }
47
48 void postorder(Node *root)
49 {
50     if (!root)
51         return;
52     stack<Node *> st1, st2;
53     st1.push(root);
54
55     while (!st1.empty())
56     {
57         Node *curr = st1.top();
58         st1.pop();
59         st2.push(curr);
60
61         if (curr->left)
62             st1.push(curr->left);
63         if (curr->right)
64             st1.push(curr->right);
65     }
66
67     while (!st2.empty())
68     {
69         cout << st2.top()->data << " ";
70         st2.pop();
71     }
72 }
73
74 int main()
75 {
76     Node *root = buildTreeFromFile("file.txt");
77     cout << "Postorder: ";
78     postorder(root);
79     cout << endl;
80     return 0;
81 }
```

Input File (file.txt):

Lab 8 > file.txt

```
1  2 78 3
2  4 79 5
3  0 80 6
4  0 76 0
5  7 77 0
6  0 75 0
7  0 71 0
8  -1 -1 -1
```

2. Binary Search Tree

```
Lab 8 > G++ bst.cpp > ...
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Node
5 {
6 public:
7     int data;
8     Node *left, *right;
9
10    Node(int val)
11    {
12        data = val;
13        left = right = NULL;
14    }
15 };
16
17 Node *insertBST(Node *root, int val)
18 {
19     if (root == NULL)
20         return new Node(val);
21
22     if (val < root->data)
23         root->left = insertBST(root->left, val);
24     else if (val > root->data)
25         root->right = insertBST(root->right, val);
26
27     return root;
28 }
29
30 Node *minValueNode(Node *node)
31 {
32     while (node && node->left != NULL)
33         node = node->left;
34     return node;
35 }
36
37 Node *deleteBST(Node *root, int val)
38 {
39     if (root == NULL)
40         return root;
41
42     if (val < root->data)
43         root->left = deleteBST(root->left, val);
44     else if (val > root->data)
45         root->right = deleteBST(root->right, val);
46     else
47     {
48         if (root->left == NULL)
49         {
50             Node *temp = root->right;
51             delete root;
52
53             delete root;
54             return temp;
55         }
56         else if (root->right == NULL)
57         {
58             Node *temp = root->left;
59             delete root;
60             return temp;
61         }
62
63         Node *temp = minValueNode(root->right);
64         root->data = temp->data;
65         root->right = deleteBST(root->right, temp->data);
66
67         return root;
68     }
69 }
70
71 void inorder(Node *root)
72 {
73     if (root != NULL)
74     {
75         inorder(root->left);
76         cout << root->data << " ";
77         inorder(root->right);
78     }
79 }
80
81 int main()
82 {
83     ifstream file("bst.txt");
84     int val;
85
86     Node *root = NULL;
87
88     while (file >> val)
89     {
90         if (val == -1)
91             break;
92         root = insertBST(root, val);
93     }
94
95     cout << "Inorder after insertion: ";
96     inorder(root);
97
98     root = deleteBST(root, 50);
99
100    cout << "\nInorder after deletion: ";
101    inorder(root);
102    return 0;
103 }
```

Input File (bst.txt):

```
Lab 8 > bst.txt
1 50 30 70 20 40 60 80 -1
```

Procedure:

1. IDE (VS Code) was opened and project created
2. Binary Tree traversal programs were executed using file input
3. Outputs of Inorder, Preorder, Postorder were verified
4. BST program was compiled and executed using separate file input
5. Insertion and deletion operations were tested
6. Outputs were verified using inorder traversal

Results:

Binary Tree:

Inorder: 76 79 71 77 78 80 75 Preorder: 78 79 76 77 71 80 75 Postorder: 76 71 77 79 75 80 78

Binary Search Tree:

Inorder after insertion: 20 30 40 50 60 70 80

Inorder after deletion: 20 30 40 60 70 80

Complexity Analysis:

Operation	Time Complexity	Space Complexity
Tree Traversal	$O(n)$	$O(1)$
BST Insertion	$O(h)$	$O(1)$
BST Deletion	$O(h)$	$O(1)$

Here, h = height of tree

Precautions:

- Ensured correct file format before execution
- Checked NULL pointers before operations
- Avoided invalid node access during deletion
- Verified output after each operation

Limitations:

- BST may degrade to $O(n)$ in worst case
- File-based structure requires careful formatting
- No direct random access like arrays

Conclusion:

This experiment successfully demonstrated Binary Tree traversal and Binary Search Tree operations. Binary Tree traversal helped understand hierarchical structure navigation, while BST operations provided insight into efficient data organization using ordered insertion and deletion. These concepts are fundamental for advanced data structures and algorithms

Experiment No: 12

Experiment Name: Insertion and Deletion on Heap, Heap Sort, Hash Table

Objectives:

- To implement insertion and deletion operations in a binary heap
- To understand Heap Sort using heap data structure
- To implement a hash table with insertion, searching, and deletion
- To analyze time complexity of heap and hashing operations

Theory:

A heap is a specialized complete binary tree-based data structure that satisfies the heap property. In a max heap, every parent node is greater than or equal to its children, while in a min heap, every parent node is smaller than or equal to its children. Heaps are widely used in priority queue implementations because they allow efficient insertion, deletion, and retrieval of the highest or lowest priority element. The structure ensures logarithmic time complexity for core operations due to its complete binary tree nature

Heap Sort is a comparison-based sorting technique that uses a binary heap to sort elements. It first builds a max heap from the input array, ensuring the largest element is at the root. Then, it repeatedly swaps the root with the last element, reduces the heap size, and restores heap property using heapify. This process continues until the array becomes fully sorted. Heap Sort is efficient and does not require additional memory for sorting, making it an in-place algorithm

A hash table is a data structure used for fast data retrieval using key-value mapping. It uses a hash function to compute an index where data is stored. When collisions occur (multiple keys mapping to the same index), techniques like linear probing are used to find the next available position. Hash tables provide near constant-time complexity for insertion, deletion, and search operations under ideal conditions, making them highly efficient for large-scale data access systems

Algorithms:

1 | Heap Insertion Algorithm:

- Insert the new element at the end of the heap
- Compare the element with its parent node
- If heap property is violated, swap with parent
- Repeat until heap property is restored
- Stop when element is correctly positioned

2 | Heap Deletion Algorithm:

- Remove the root element (maximum element in max heap)

- Replace root with last element in heap
- Reduce heap size by one
- Apply heapify operation from root
- Restore heap property recursively

3 | Heap Sort Algorithm:

- Build a max heap from the input array
- Swap root element with last element
- Reduce heap size
- Apply heapify to restore heap property
- Repeat until only one element remains

4 | Hash Table Operations Algorithm:

- **Insertion:** Compute hash index using hash function, insert element; if collision occurs, use linear probing to find next empty slot
- **Deletion:** Search for key using probing and mark the slot as deleted (-1)
- **Search:** Compute hash index and check for key; if not found, continue probing until element is found or table ends

Code:

Heap Operations & Heap Sort:

```
G++ heap.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  #define MAX 100
5
6  int heap[MAX];
7  int heap_size = 0;
8
9  void swap(int &a, int &b)
10 {
11     int temp = a;
12     a = b;
13     b = temp;
14 }
15
16 void heapify_up(int i)
17 {
18     while (i > 0 && heap[(i - 1) / 2] < heap[i])
19     {
20         swap(heap[i], heap[(i - 1) / 2]);
21         i = (i - 1) / 2;
22     }
23 }
24
25 void heapify_down(int i)
26 {
27     int largest = i;
28     int left = 2 * i + 1;
29     int right = 2 * i + 2;
30
31     if (left < heap_size && heap[left] > heap[largest])
32         largest = left;
33
34     if (right < heap_size && heap[right] > heap[largest])
35         largest = right;
36
37     if (largest != i)
38     {
39         swap(heap[i], heap[largest]);
40         heapify_down(largest);
41     }
42 }
43
44 void insert_heap(int val)
45 {
46     heap[heap_size] = val;
47     heap_size++;
48     heapify_up(heap_size - 1);
49 }
50
51 void delete_heap()
52 {
53     if (heap_size <= 0)
54     {
55         cout << "Heap empty!\n";
56         return;
57     }
58
59     heap[0] = heap[heap_size - 1];
60     heap_size--;
61     heapify_down(0);
62 }
63
64 void print_heap()
65 {
66     for (int i = 0; i < heap_size; i++)
67         cout << heap[i] << " ";
68     cout << endl;
69 }
70
71 int main()
72 {
73     int choice, val;
74
75     while (true)
76     {
77         cout << "\n--- Heap Operations ---\n";
78         cout << "1. Insert\n2. Delete\n3. Print\n4. Exit\n";
79         cout << "Enter choice: ";
80         cin >> choice;
81
82         switch (choice)
83         {
84             case 1:
85                 cout << "Enter value: ";
86                 cin >> val;
87                 insert_heap(val);
88                 break;
89
90             case 2:
91                 delete_heap();
92                 break;
93
94             case 3:
95                 print_heap();
96                 break;
97
98             case 4:
99                 return 0;
100
101             default:
102                 cout << "Invalid choice!\n";
103         }
104     }
105 }
```

Hash Table:

```
G++ hash.cpp > main()
1  #include <iostream>
2  using namespace std;
3
4  #define TABLE_SIZE 10
5
6  int hash_table[TABLE_SIZE];
7
8  int hash_func(int key)
9  {
10     return key % TABLE_SIZE;
11 }
12
13 void insert_hash(int key)
14 {
15     int index = hash_func(key);
16     int start = index;
17
18     while (hash_table[index] != -1)
19     {
20         index = (index + 1) % TABLE_SIZE;
21         if (index == start)
22         {
23             cout << "Table full!\n";
24             return;
25         }
26     }
27     hash_table[index] = key;
28 }
29
30 int search_hash(int key)
31 {
32     int index = hash_func(key);
33     int start = index;
34
35     while (hash_table[index] != -1)
36     {
37         if (hash_table[index] == key)
38             return index;
39         index = (index + 1) % TABLE_SIZE;
40         if (index == start)
41             break;
42     }
43     return -1;
44 }
45
46 void delete_hash(int key)
47 {
48     int idx = search_hash(key);
49     if (idx != -1)
50         hash_table[idx] = -1;
51     else
52         cout << "Not found!\n";
53 }
54
55 void display()
56 {
57     for (int i = 0; i < TABLE_SIZE; i++)
58         cout << i << " -> " << hash_table[i] << endl;
59 }
60
61
62
63 }
64
65 int main()
66 {
67     for (int i = 0; i < TABLE_SIZE; i++)
68         hash_table[i] = -1;
69
70     int choice, val;
71
72     while (true)
73     {
74         cout << "\n--- Hash Table Operations ---\n";
75         cout << "1. Insert\n2. Search\n3. Delete\n4. Display\n5. Exit\n";
76         cout << "Enter choice: ";
77         cin >> choice;
78
79         switch (choice)
80         {
81             case 1:
82                 cout << "Enter value: ";
83                 cin >> val;
84                 insert_hash(val);
85                 break;
86             case 2:
87                 cout << "Enter value: ";
88                 cin >> val;
89                 if (search_hash(val) != -1)
90                     cout << "Found!\n";
91                 else
92                     cout << "Not found!\n";
93                 break;
94             case 3:
95                 cout << "Enter value: ";
96                 cin >> val;
97                 delete_hash(val);
98                 break;
99             case 4:
100                 display();
101                 break;
102             case 5:
103                 return 0;
104             default:
105                 cout << "Invalid choice!\n";
106         }
107     }
108 }
109
110
111
112
113 }
```

Procedure:

- Created two separate C++ programs for heap operations and hash table implementation
- Implemented insertion, deletion, and heapify operations for maintaining heap property
- Tested heap operations with multiple inputs and verified structure consistency
- Implemented hash table using linear probing for collision handling
- Tested insertion, search, deletion, and display operations in hash table

Results:

--- Heap Operations ---	--- Heap Operations ---
1. Insert	1. Insert
2. Delete	2. Delete
3. Print	3. Print
4. Exit	4. Exit
Enter choice: 1	Enter choice: 3
Enter value: 179	179

```

--- Hash Table Operations ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 1
Enter value: 179

--- Hash Table Operations ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 4
0 -> -1
1 -> -1
2 -> -1
3 -> -1
4 -> -1
5 -> -1
6 -> -1
7 -> -1
8 -> -1
9 -> 179

```

Complexity Analysis:

Operation	Heap	Hash Table
Insertion	$O(\log n)$	$O(1)$ avg, $O(n)$ worst
Deletion	$O(\log n)$	$O(1)$ avg, $O(n)$ worst
Search	$O(n)$	$O(1)$ avg, $O(n)$ worst
Heap Sort	$O(n \log n)$	N/A

Precautions:

- Ensure heap is not empty before performing deletion
- Maintain heap property after every operation
- Initialize hash table with -1 to indicate empty slots
- Handle collisions properly using linear probing
- Avoid array overflow and invalid indexing

Conclusion:

This experiment demonstrates the practical implementation of heap operations, heap sort, and hash tables. It enhances understanding of priority-based data structures and fast lookup systems. Heap structures provide efficient priority management, while hash tables enable near constant-time data retrieval, making both essential tools in algorithm design and system optimization.